WK 02: SQL-Tables

Working with Relational Data

PPOL 5206 | Massive Data Fundamentals | Spring 2026

Before SQL: The Data Problem

Why Do We Need Structure?

IMAGINE YOU'RE RUNNING A CAT CAFE...

You need to track:

- Which cats are available for adoption
- Which customers visited and when
- What interactions happened (lap time, play sessions)
- Staff schedules and assignments
- Menu items and drink orders

Your first instinct: Put everything in one big spreadsheet.
What could possibly go wrong?

THE SINGLE-SPREADSHEET APPROACH

Common first instinct: Put everything in one big spreadsheet.

What could possibly go wrong?

visit_id	customer	email	cats
1	Maya Chen	maya@email.com	Whiskers, Mittens
2	Maya Chen	maya@email.com	Whiskers
3	Alex Rivera	alex@email.com	Shadow

Problems emerge quickly...

WHAT GOES WRONG: UPDATE ANOMALY

UPDATE ANOMALY

Maya changes her phone number from 555-0101 to 555-9999.

- How many rows need updating?
- What if you miss one?
- Now you have inconsistent data!

WHAT GOES WRONG: MORE ANOMALIES

DELETE ANOMALY

Alex cancels his visit and you delete that row.

You just lost the fact that a cat named Shadow exists!

INSERT ANOMALY

You adopt a new cat named "Cleo" from a shelter.

Can you add her before anyone visits her?

> These problems have names because they happen **ALL THE TIME.**

WHAT GOES WRONG: MULTI-VALUE PROBLEM

"Whiskers, Mittens" stored in one cell.

How do you query "show me all visits involving Whiskers"?

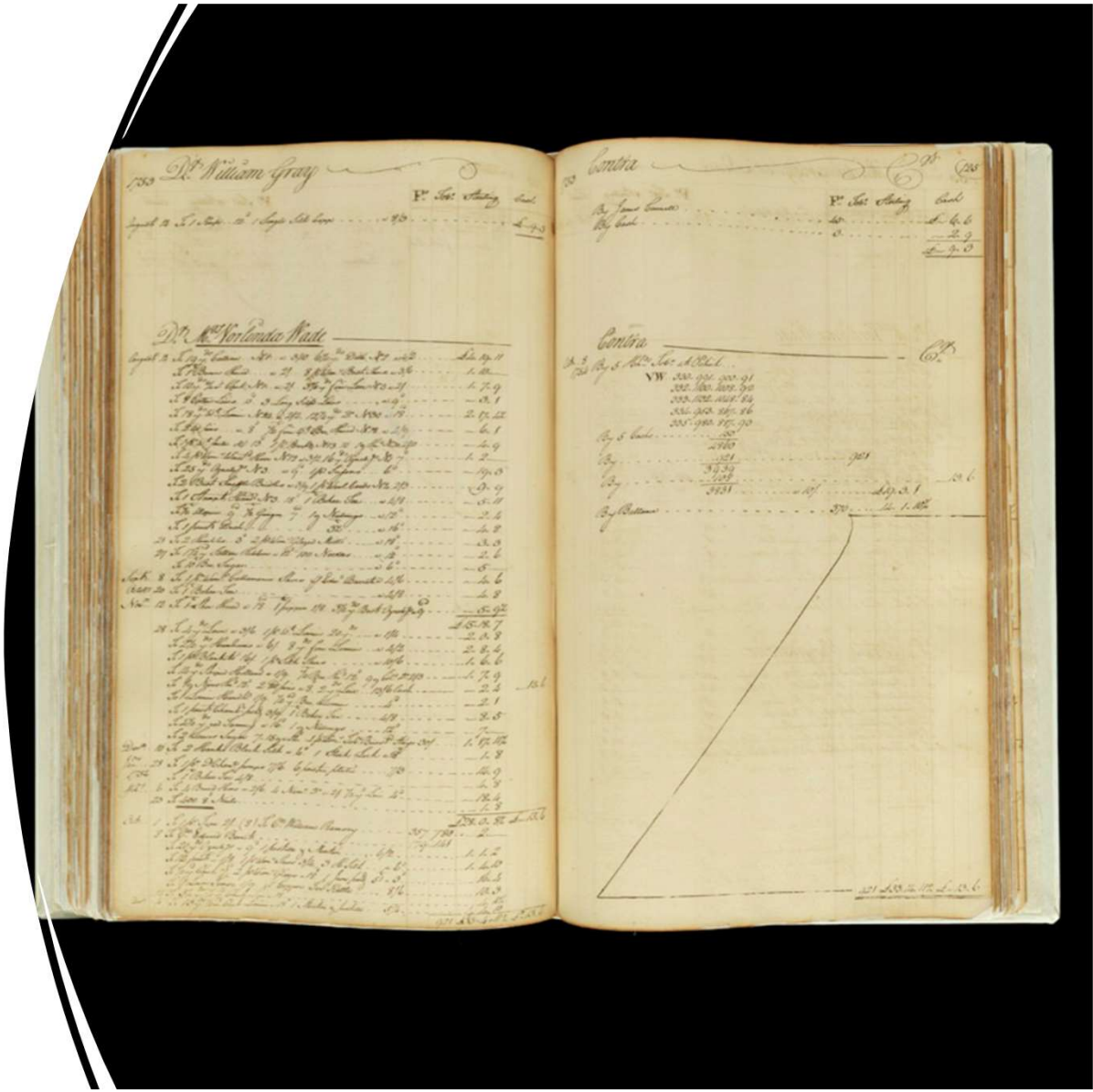
You'd have to:

- Parse the string to find "Whiskers"
- Handle different separators (comma, semicolon, etc.)
- Deal with spaces, typos, variations

This is fragile and slow.

DATA: 1756

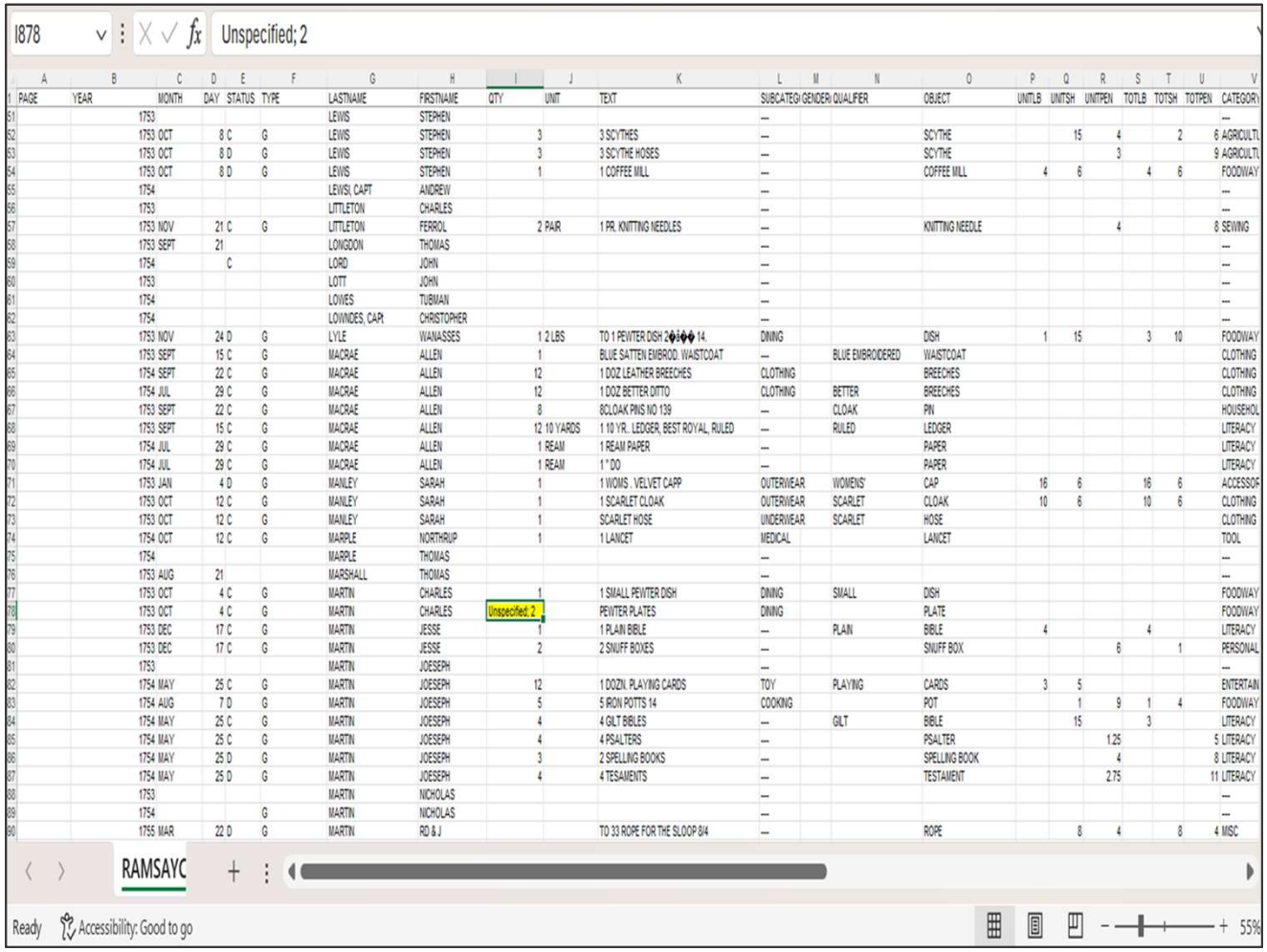
- Source: [Baddledors, twig whips, and yards of thunder and lightning: Decoding a colonial ledger | National Museum of American History (si.edu)]



DATA: 2013

The same fundamental structure
(rows and columns)
persists across 250+ years.

Source: Ramsay Ledger (wisc.edu)



I878 | Unspecified; 2

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
1	PAGE	YEAR	MONTH	DAY	STATUS	TYPE	LASTNAME	FIRSTNAME	QTY	UNIT	TEXT	SUBCATEGI	GENDER	QUALIFER	OBJECT	UNITLB	UNITSH	UNITPEN	TOTLB	TOTSH	TOTPEN	CATEGORY
51			1753				LEWIS	STEPHEN				---										
52			1753	OCT	8	C	G	LEWIS	STEPHEN	3		3 SCYTHES	---			SCYTHE			15	4		2	6 AGRICULTU
53			1753	OCT	8	D	G	LEWIS	STEPHEN	3		3 SCYTHE HOSES	---			SCYTHE				3			9 AGRICULTU
54			1753	OCT	8	D	G	LEWIS	STEPHEN	1		1 COFFEE MILL	---			COFFEE MILL	4	6		4	6		FOODWAY
55			1754					LEWSI, CAPT	ANDREW				---										
56			1753					LITTLETON	CHARLES				---										
57			1753	NOV	21	C	G	LITTLETON	FERROL	2	PAIR	1 PR. KNITTING NEEDLES	---			KNITTING NEEDLE				4			8 SEWING
58			1753	SEPT	21			LONGDON	THOMAS				---										
59			1754			C		LORD	JOHN				---										
60			1753					LOTT	JOHN				---										
61			1754					LOWES	TUBMAN				---										
62			1754					LOWNDES, CAPT	CHRISTOPHER				---										
63			1753	NOV	24	D	G	LYLE	WANASSES	1	2 LBS	TO 1 PEWTER DISH 2 ♦ ♦ 14.	DINING			DISH	1	15		3	10		FOODWAY
64			1753	SEPT	15	C	G	MACRAE	ALLEN	1		BLUE SATTEN EMBROD. WAISTCOAT	---		BLUE EMBRODERED	WAISTCOAT							CLOTHING
65			1754	SEPT	22	C	G	MACRAE	ALLEN	12		1 DOZ LEATHER BREECHES	CLOTHING			BREECHES							CLOTHING
66			1754	JUL	29	C	G	MACRAE	ALLEN	12		1 DOZ BETTER DITTO	CLOTHING		BETTER	BREECHES							CLOTHING
67			1753	SEPT	22	C	G	MACRAE	ALLEN	8		8CLOAK PINS NO 139	---		CLOAK	PIN							HOUSEHOL
68			1753	SEPT	15	C	G	MACRAE	ALLEN	12	10 YARDS	1 10 YR. LEDGER, BEST ROYAL, RULED	---		RULED	LEDGER							LITERACY
69			1754	JUL	29	C	G	MACRAE	ALLEN	1	REAM	1 REAM PAPER	---			PAPER							LITERACY
70			1754	JUL	29	C	G	MACRAE	ALLEN	1	REAM	1 " DO	---			PAPER							LITERACY
71			1753	JAN	4	D	G	MANLEY	SARAH	1		1 WOMS . VELVET CAPP	OUTERWEAR		WOMENS'	CAP	16	6		16	6		ACCESSOF
72			1753	OCT	12	C	G	MANLEY	SARAH	1		1 SCARLET CLOAK	OUTERWEAR		SCARLET	CLOAK	10	6		10	6		CLOTHING
73			1753	OCT	12	C	G	MANLEY	SARAH	1		SCARLET HOSE	UNDERWEAR		SCARLET	HOSE							CLOTHING
74			1754	OCT	12	C	G	MARPLE	NORTHRUP	1		1 LANCET	MEDICAL			LANCET							TOOL
75			1754					MARPLE	THOMAS				---										
76			1753	AUG	21			MARSHALL	THOMAS				---										
77			1753	OCT	4	C	G	MARTIN	CHARLES	1		1 SMALL PEWTER DISH	DINING		SMALL	DISH							FOODWAY
78			1753	OCT	4	C	G	MARTIN	CHARLES	Unspecified; 2		PEWTER PLATES	DINING			PLATE							FOODWAY
79			1753	DEC	17	C	G	MARTIN	JESSE	1		1 PLAIN BIBLE	---		PLAIN	BIBLE	4			4			LITERACY
80			1753	DEC	17	C	G	MARTIN	JESSE	2		2 SNUFF BOXES	---			SNUFF BOX			6		1		PERSONAL
81			1753					MARTIN	JOESEPH				---										
82			1754	MAY	25	C	G	MARTIN	JOESEPH	12		1 DOZN. PLAYING CARDS	TOY		PLAYING	CARDS	3	5					ENTERTAIN
83			1754	AUG	7	D	G	MARTIN	JOESEPH	5		5 IRON POTTS 14	COOKING			POT		1	9	1	4		FOODWAY
84			1754	MAY	25	C	G	MARTIN	JOESEPH	4		4 GILT BIBLES	---		GILT	BIBLE		15		3			LITERACY
85			1754	MAY	25	C	G	MARTIN	JOESEPH	4		4 PSALTERS	---			PSALTER			1.25				5 LITERACY
86			1754	MAY	25	D	G	MARTIN	JOESEPH	3		2 SPELLING BOOKS	---			SPELLING BOOK			4				8 LITERACY
87			1754	MAY	25	D	G	MARTIN	JOESEPH	4		4 TESAMENTS	---			TESTAMENT			2.75				11 LITERACY
88			1753					MARTIN	NICHOLAS				---										
89			1754				G	MARTIN	NICHOLAS				---										
90			1755	MAR	22	D	G	MARTIN	RD & J			TO 33 ROPE FOR THE SLOOP 8/4	---			ROPE		8	4		8	4	MISC

RAMSAYC

Ready | Accessibility: Good to go | 55%

THE RELATIONAL DATABASE SOLUTION

Instead of one messy table, use multiple connected tables

TABLE = An entity type (cats, customers, visits)

ROW = One specific instance (Whiskers, Maya Chen)

COLUMN = One attribute (name, email, visit_date)

PRIMARY KEYS & FOREIGN KEYS

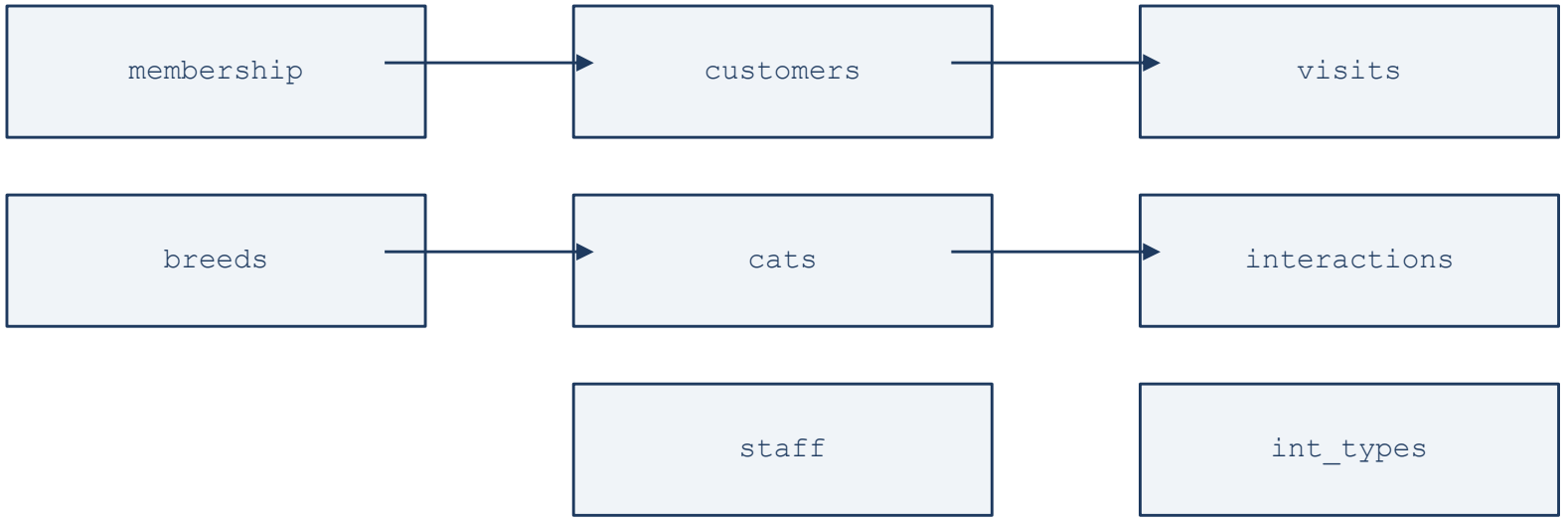
cat_id	name	breed_id	status
1	Whiskers	1	available
2	Mittens	2	available
3	Shadow	2	adopted

PRIMARY KEY: cat_id (uniquely identifies each row)

FOREIGN KEY: breed_id (links to the "breeds" table)

THE WHISKERS & WAFFLES DATABASE

7 tables:



Each entity in its own table. Relationships through keys.

ACID Properties

Ensure data integrity:

1. Atomicity
2. Consistency
3. Isolation
4. Durability

ACID: ATOMICITY

Each transaction succeeds completely or fails completely.

Example: When someone adopts a cat:

```
1. Update cat.status to 'adopted'
2. Set cat.adoption_date
3. Create an adoption record
4. Process payment
```

If step 3 fails → ALL changes roll back.

ACID: CONSISTENCY & ISOLATION

CONSISTENCY

A transaction can only bring the database from one valid state to another.

ISOLATION

Transactions are processed independently.

Two staff can't simultaneously mark the same cat as adopted by different people.

ACID: DURABILITY

Once a transaction is committed, it remains permanent.

Even if the power goes out.

That adoption record from 2019? Still there.

Introduction to SQL

The Language of Relational Data

SQL: ASKING QUESTIONS OF DATA

SQL lets us ASK QUESTIONS of structured databases:

"Which cats are available for adoption?"

"How many visits did Maya Chen make in March?"

"What's the most popular interaction type?"

"Which staff member has supervised the most visits?"

KEY CONCEPT: DECLARATIVE PROGRAMMING

You tell the database **WHAT** you want
not **HOW** to get it.

The database engine's query optimizer figures out the most efficient path.

SQL: PRONUNCIATION & HISTORY

Pronounced "sequel" or "S-Q-L" both are correct

- Originally named SEQUEL (Structured English Query Language)
- Shortened to SQL due to trademark conflict
- Many people kept saying "sequel" anyway

- **Fun fact:** ISO standard explicitly states SQL is NOT an acronym.
- *"SQL is a language name, not an acronym." — ISO/IEC 9075*

- SQL is a standard (ANSI, ISO)
- But there are various brands that have their own syntax
    - MySQL, PostgreSQL, etc.

SQL: SOME HISTORY

Era	Event
Pre-1970	Hierarchical/network databases dominate. Difficult to access data.
1970	Ted Codd (IBM) describes the Relational Model
1970s	IBM develops SEQUEL → shortened to SQL
1979	Oracle releases first commercial RDBMS
1983	IBM releases DB2
1986-87	SQL standardized by ANSI, then ISO
1989	Microsoft and Sybase release SQL Server (for OS/2)
1996	PostgreSQL (open source) released

WHY LEARN SQL?

You may already use SQL without knowing it:

Retrieving grades online?
SQL
Your company's reporting dashboard?
SQL
Government data systems?
SQL

SQL SPANS INDUSTRIES & GENERATIONS

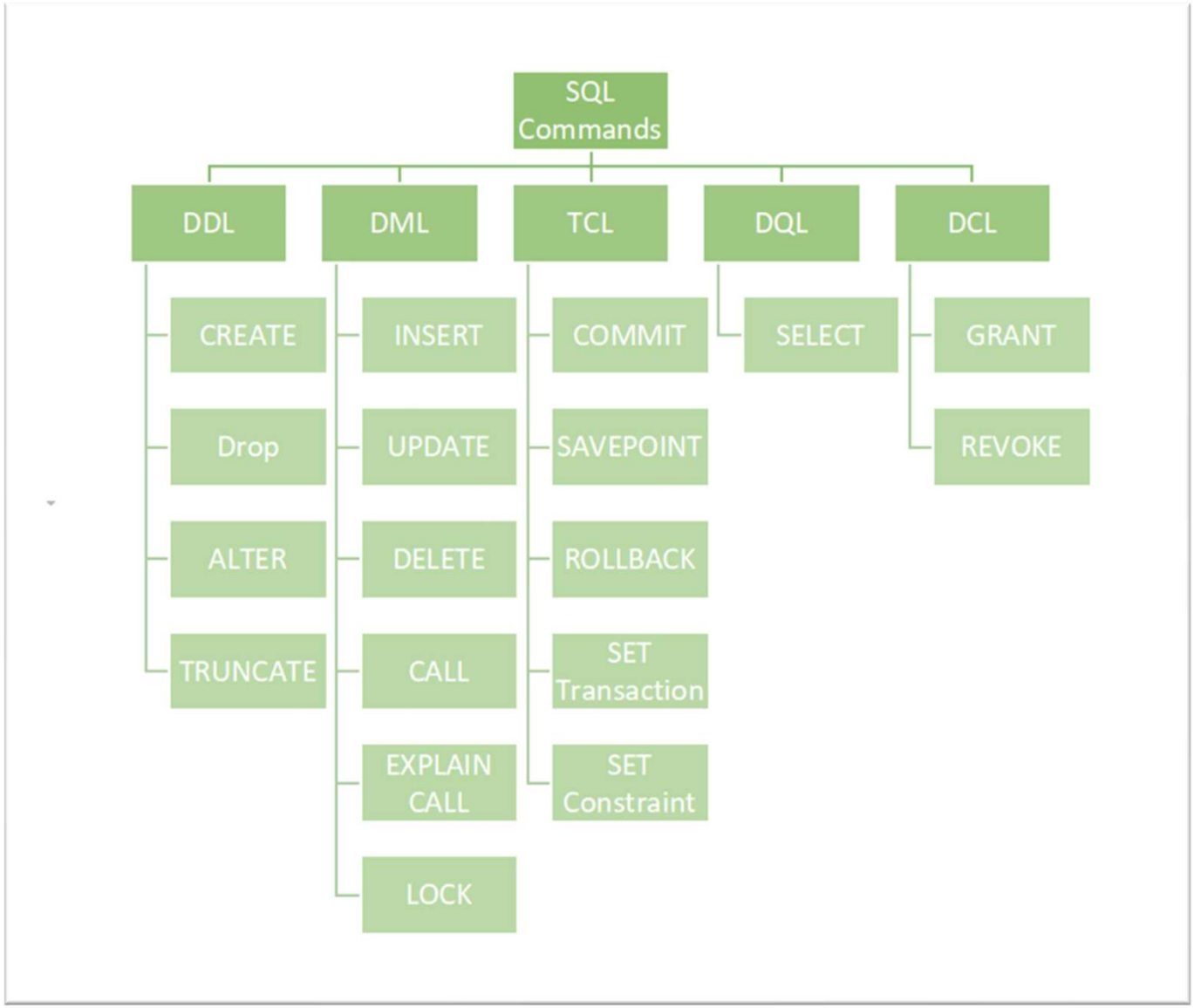
- Not just big tech: hospitals, local governments, nonprofits all use SQL
- Skills transfer across sectors
- This is a 30+ year career skill, not a fad

The ubiquitous nature of SQL makes it one of the most versatile and portable tech skills you can acquire.

SQL "LANGUAGES"

Language	Purpose	Commands
DDL	Data Definition Language	CREATE, ALTER, DROP
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE
DQL	Data Query Language	SELECT (sometimes separate)
DCL	Data Control Language	GRANT, REVOKE
TCL	Transaction Control Language	COMMIT, ROLLBACK

For this course: We focus on DML/DQL querying and manipulating data.


SQL Commands
DDL
DML
TCL
DQL
DCL
CREATE
Drop
ALTER
TRUNCATE
INSERT
UPDATE
DELETE
CALL
EXPLAIN CALL
LOCK
COMMIT
SAVEPOINT
ROLLBACK
SET Transaction
SET Constraint
SELECT
GRANT
REVOKE

DDL VS DML

DDL — Defines STRUCTURE

Think: "Building the filing cabinet"

```
CREATE TABLE cats (
  cat_id INTEGER,
  name VARCHAR(50)
);
```

DML — Works with DATA

Think: "Using the filing cabinet"

```
SELECT name FROM cats
WHERE status = 'available';
```

DDL:

- Often permanent changes, affects entire tables
- Used rarely

DML:

- Day-to-day operations, affects rows,
- Used constantly

CRUD: FOUR FUNDAMENTAL OPERATIONS

Operation	SQL Command	Example
CREATE	`INSERT`	Add a new cat
READ	`SELECT`	Find available cats
UPDATE	`UPDATE`	Mark cat as adopted
DELETE	`DELETE`	Remove old visits

Every application you've ever used performs these four operations.

CRUD: THE FOUR FUNDAMENTAL OPERATIONS

CREATE (INSERT)

```
INSERT INTO cats (name,
breed_id)
VALUES ('Cleo', 3);
```

READ (SELECT)

```
SELECT * FROM cats
WHERE status = 'available';
```

UPDATE

```
UPDATE cats
SET status = 'adopted'
WHERE cat_id = 3;
```

DELETE

```
DELETE FROM visits
WHERE visit_date < '2023-01-
01';
```

SQL Data Types

Choosing the Right Container

COMMON SQL DATA TYPES

TYPE	DESCRIPTION	CAT CAFE EXAMPLE
INTEGER	Whole numbers	cat_id = 7
VARCHAR(N)	Variable text up to N chars	name = 'Whiskers'
TEXT	Unlimited text	notes = 'Very playful...'
DECIMAL(p,s)	Precise decimals	price = 25.00
BOOLEAN	True/False	is_neutered = TRUE
DATE	Calendar date	arrival_date = '2024-03-15'
TIMESTAMP	Date + Time	check_in = '2024-03-15 14:30:00'

DATE/TIME HANDLING

Dates are consistently confusing — but essential for policy work

Extract parts of dates:

```
SELECT
  EXTRACT(YEAR FROM visit_date) AS year,
  EXTRACT(MONTH FROM visit_date) AS month
FROM visits;
```

DATE ARITHMETIC

Visits in the last 30 days:

```
SELECT * FROM visits
WHERE visit_date ⩾ CURRENT_DATE - 30;
```

Age calculation:

```
SELECT name, AGE(CURRENT_DATE,
birth_date)
FROM cats;
```

NULL: THE ABSENCE OF DATA

NULL means "unknown" or "not applicable"

NOT zero. NOT empty string.

first_name	phone
Maya	555-0101
Alex	555-0202
Taylor	*NULL*

FINDING NULL VALUES

Use IS NULL or IS NOT NULL:

```
SELECT * FROM customers
WHERE phone IS NULL;
```

COMMON MISTAKE:
WHERE phone = NULL

NEVER matches!
Use IS NULL instead.

COALESCE: PROVIDING DEFAULT VALUES

Returns the first non-NULL value in a list:

```
SELECT
  first_name,
  COALESCE(phone, 'No phone on file') AS
contact
FROM customers;
```

first_name	contact
Maya	555-0101
Taylor	No phone on file

COALESCE: MORE USES

Multiple fallbacks:

```
SELECT COALESCE(nickname, first_name, 'Guest')
FROM customers;
```

Default to 0 for calculations:

```
SELECT SUM(COALESCE(tip_amount, 0))
FROM visits;
```

COALESCE is useful in reports where NULL values would be confusing.

STRING FUNCTIONS: CONCAT

Join strings together:

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM customers;
```

full_name
Maya Chen
Alex Rivera
Taylor Nguyen

STRING FUNCTIONS: UPPER, LOWER, TRIM

UPPER / LOWER:

```
SELECT UPPER(name) FROM cats;
-- Returns 'WHISKERS'
```

TRIM:

```
SELECT TRIM('  Whiskers  ');
-- Returns 'Whiskers'
```

LENGTH:

```
SELECT name, LENGTH(name) FROM cats;
```

The BIG SIX SQL Clauses

The Foundation of Every Query

SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY

THE BIG SIX: from QUESTIONS to SQL

QUESTION	CLAUSE	CAT CAFE EXAMPLE
What data do I want?	`SELECT`	cat name, breed, status
Where does it come from?	`FROM`	cats table
Which rows?	`WHERE`	only available cats
How should rows be grouped?	`GROUP BY`	count by breed
Which groups to keep?	`HAVING`	breeds with 2+ cats
What order for results?	`ORDER BY`	alphabetically by name

SELECT: CHOOSING YOUR COLUMNS

```
SELECT name, status FROM cats;
```

name	status
Whiskers	available
Mittens	available
Shadow	adopted

Select everything (use sparingly):

```
SELECT * FROM cats;
```

> Tip: Avoid SELECT * in production: it's slower and returns more data than needed.

SELECT DISTINCT: REMOVING DUPLICATES

Problem: Maya visited multiple times

```
SELECT customer_id FROM
visits;
```

Returns: 1, 1, 2, 3, 4, 1, 5, 2 (duplicates!)

```
SELECT DISTINCT customer_id FROM
visits;
```

Returns: 1, 2, 3, 4, 5 (unique values only)

AGGREGATE FUNCTIONS

Function	Question	Example
`COUNT(*)`	How many?	How many cats?
`SUM()`	What's the total?	Total interaction minutes
`AVG()`	What's the average?	Average visit duration
`MIN()`	Smallest value?	First arrival date
`MAX()`	Largest value?	Most recent visit

AGGREGATE FUNCTIONS: SUMMARIZING DATA

`COUNT(*)` — How many cats are available?

```
SELECT COUNT(*) FROM cats
WHERE status = 'available';  → 5
```

`SUM()` — Total interaction minutes?

```
SELECT SUM(duration_mins)
FROM interactions;  → 327
```

`AVG()` — Average interaction duration?

```
SELECT AVG(duration_mins)
FROM interactions;  → 27.5
```

`MIN()` / `MAX()` — Oldest & newest arrivals?

```
SELECT MIN(arrival_date),
       MAX(arrival_date) FROM cats;
```

These functions collapse many rows into one summary value.

COUNT: COUNTING ROWS

```
SELECT COUNT(*) FROM cats
WHERE status = 'available';
```

Result: 5

first_name	phone
Maya	555-0101
Alex	555-0202
Sam	555-0303
Taylor	*NULL*
Jordan	555-0505

COUNT(*) vs COUNT(column):

```
COUNT(*)      -- Counts ALL rows: 5
COUNT(phone)  -- Counts non-NULL values: 4
```

`Count(*)` and `Count(1)`
should return the same results

ALIASES: RENAMING FOR CLARITY

```
SELECT
  c.name AS cat_name,
  COUNT(i.interaction_id) AS
total_interactions
FROM cats c
JOIN interactions i ON c.cat_id = i.cat_id
GROUP BY c.cat_id, c.name;
```

cat_name	total_interactions
Whiskers	4
Mittens	2

Table aliases *(c, i) shorten your code.*
AS *keyword improves readability.*

LIMIT: CAPPING YOUR RESULTS

```
SELECT * FROM cats
ORDER BY arrival_date DESC
LIMIT 5;
```

Returns only the 5 most recently arrived cats.

Why use LIMIT?

Preview data without loading everything
Get "top N" results
Control output size for reports

FROM: CHOOSING YOUR DATA SOURCE

Single table:

```
SELECT * FROM cats;
```

Multiple tables (joins):

```
SELECT * FROM cats c
JOIN breeds b ON c.breed_id = b.breed_id;
```

We will cover joins in detail next week in SQL 2 - Databases

WHERE: FILTERING ROWS

Basic comparisons:

```
SELECT * FROM cats
WHERE status = 'available';
```

Multiple conditions:

```
SELECT * FROM cats
WHERE status = 'available' AND breed_id = 1;
```

WHERE: MORE OPERATORS

IN for lists:

```
WHERE membership_id IN (3, 4);
```

LIKE for patterns:

```
WHERE name LIKE 'M%';
```

BETWEEN for ranges:

```
WHERE visit_date BETWEEN '2024-03-15' AND '2024-03-20';
```

% = any characters | _ = single character | BETWEEN is inclusive

CASE: CONDITIONAL LOGIC

```
SELECT name,
  CASE
    WHEN birth_date > '2023-01-01' THEN 'Kitten'
    WHEN birth_date > '2020-01-01' THEN 'Adult'
    ELSE 'Senior'
  END AS age_category
FROM cats;
```

name	age_category
Whiskers	Adult
Mochi	Kitten
Thor	Senior

GROUP BY: AGGREGATING BY CATEGORY

How many cats of each status?

```
SELECT status, COUNT(*) AS cat_count
FROM cats
GROUP BY status;
```

status	cat_count
available	5
adopted	1
medical_hold	1

GROUP BY: CRITICAL RULE

> When using aggregate functions (COUNT, SUM, AVG),
> every non-aggregated column in SELECT must appear in GROUP BY.

This query will FAIL:

```
SELECT first_name, last_name, COUNT(*)
FROM customers
GROUP BY last_name;  -- first_name missing!
```

HAVING: FILTERING GROUPS

WHERE filters ROWS (before grouping)

HAVING filters GROUPS (after grouping)

```
SELECT breed_name, COUNT(*) AS cat_count
FROM cats c
JOIN breeds b ON c.breed_id = b.breed_id
GROUP BY breed_name
HAVING COUNT(*) > 1;
```

ORDER BY: SORTING RESULTS

Ascending (default):

```
ORDER BY name;
```

Descending:

```
ORDER BY arrival_date DESC;
```

Multiple columns:

```
ORDER BY status ASC, arrival_date DESC;
```

Data Manipulation

INSERT, UPDATE, DELETE

INSERT: ADDING NEW DATA

Add a single row:

```
INSERT INTO cats (name, breed_id, status)
VALUES ('Cleo', 3, 'available');
```

Add multiple rows:

```
INSERT INTO cats (name, breed_id, status)
VALUES
  ('Luna', 2, 'available'),
  ('Mochi', 3, 'available');
```

UPDATE: MODIFYING DATA

```
UPDATE cats
SET status = 'adopted', adoption_date = '2024-02-28'
WHERE cat_id = 3;
```

⚠️ ALWAYS use WHERE with UPDATE

Without WHERE, you update EVERY row in the table!

DELETE: REMOVING DATA

```
DELETE FROM visits
WHERE visit_date < '2023-01-01';
```

⚠️ ALWAYS use WHERE with DELETE

DELETE FROM visits; **DANGER: All visits gone!**

SQL Execution Order

Written Order ≠ Processing Order

SQL: WRITTEN VS EXECUTION ORDER

You Write:	Database Processes:
1. SELECT	1. FROM / JOIN
2. FROM	2. WHERE
3. WHERE	3. GROUP BY
4. GROUP BY	4. HAVING
5. HAVING	5. SELECT
6. ORDER BY	6. ORDER BY / LIMIT

SELECT runs LATE!

```sql
SELECT customer_ID, SUM(total_amount) AS "Total"
FROM orders
WHERE order_date BETWEEN '2022-01-01' AND '2022-03-31'
AND customer_city = 'New York'
GROUP BY customer_id
ORDER BY Total DESC;
```

Order of Execution

Subqueries & CTEs

Building Complex Queries

SUBQUERIES: QUERIES WITHIN QUERIES

A subquery is a SELECT inside another statement:

```
SELECT name FROM cats
WHERE cat_id IN (
  SELECT cat_id FROM interactions
  WHERE duration_mins > 30
);
```

"Find cats who've had long interactions"

CTES: COMMON TABLE EXPRESSIONS

WITH clause makes complex queries readable:

```
WITH popular_cats AS (
  SELECT cat_id, COUNT(*) AS interactions
  FROM interactions GROUP BY cat_id
  HAVING COUNT(*) > 3
)
SELECT c.name, pc.interactions
FROM popular_cats pc
JOIN cats c ON pc.cat_id = c.cat_id;
```

CTES: WHY USE THEM?

- Named result sets — self-documenting
- Can reference earlier CTEs
- Easier to debug than nested subqueries
- Often more efficient (computed once)

VIEWS: VIRTUAL TABLES

A view is a saved query that acts like a table:

```
CREATE VIEW available_cats AS
SELECT name, breed_id, arrival_date
FROM cats WHERE status = 'available';
```

Then use it like a regular table:

```
SELECT * FROM available_cats;
```

VIEWS: WHY USE THEM?

- Simplify complex queries
- Hide complexity from users
- Security: limit data access
- Consistent calculations across reports

SQL Tools

PostgreSQL, pgAdmin, DBeaver

SQL Engine vs SQL IDE

SQL ENGINE — The "brain"

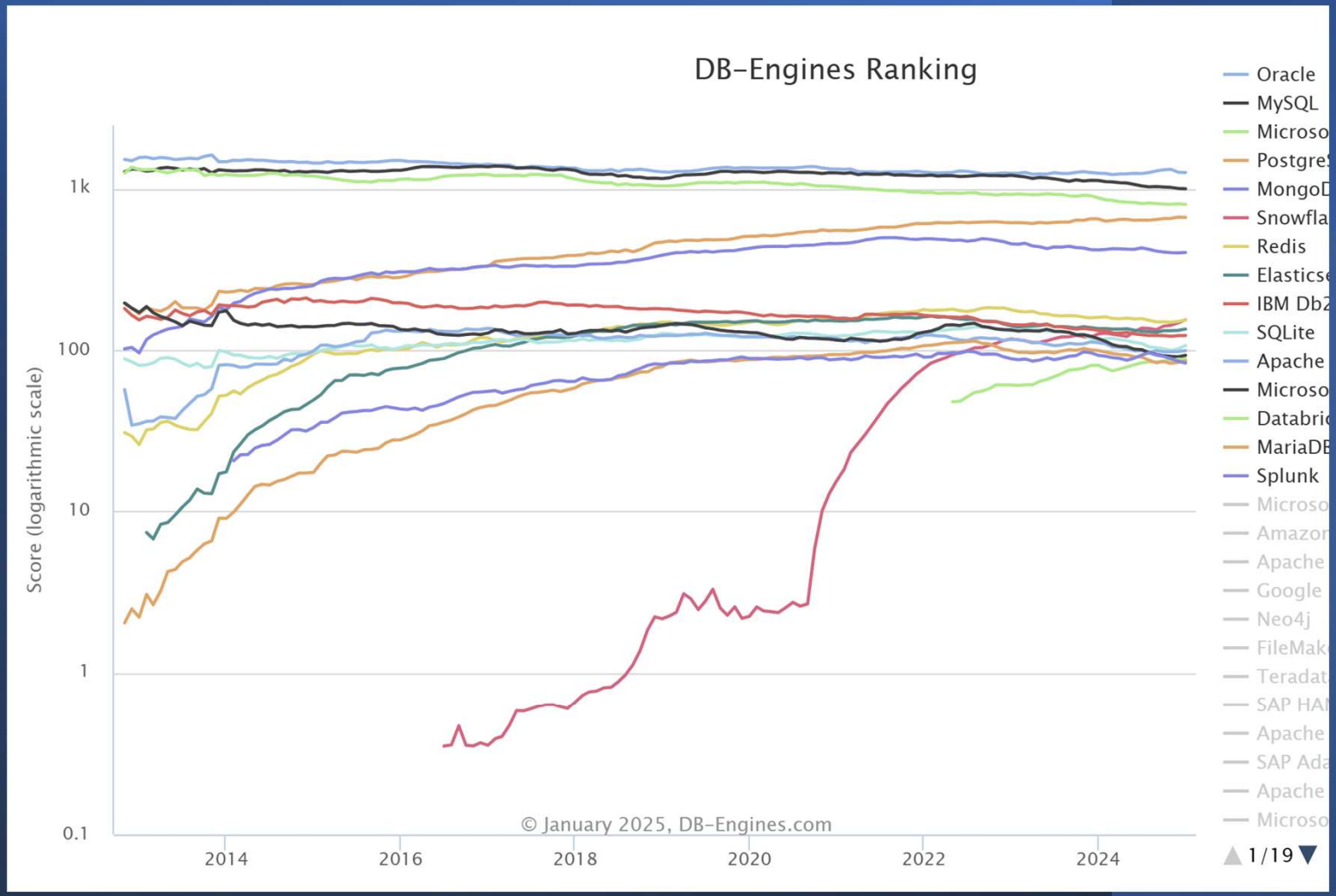
- Stores and retrieves data
- Executes queries
- Handles transactions

Examples: PostgreSQL, MySQL, Oracle

SQL IDE — The "interface"

- Write and run queries
- Explore database structure
- View results in tables

Examples: pgAdmin, DBeaver, DataGrip


DB-Engines Ranking
Score (logarithmic scale)
1k
100
10
1
0.1
2014
2016
2018
2020
2022
2024
© January 2025, DB-Engines.com
Oracle
MySQL
Microso
PostgreS
MongoD
Snowfla
Redis
Elasticse
IBM Db2
SQLite
Apache
Microso
Databric
MariaDB
Splunk
Microso
Amazon
Apache
Google
Neo4j
FileMak
Teradat
SAP HA
Apache
SAP Ada
Apache
Microso
1/19

POSTGRESQL: Used in this Course

Why PostgreSQL?

- Open source:
  - free, no licensing costs
- Feature-rich:
  - supports advanced SQL
- Extensible:
  - add capabilities (pgvector for AI)
- Enterprise-ready:
  - Apple, Instagram, Spotify

PGADMIN: POSTGRESQL'S NATIVE TOOL

- Web-based interface (runs in browser)
- Query tool with syntax highlighting
- Visual database explorer
- Server administration features

Good for: PostgreSQL-specific work, administration

DBEAVER: A Universal Database Tool

- Works with ANY database
- Desktop application (more responsive)
- ER diagrams built-in
- SQL formatter and code completion

We'll use DBeaver in lab: one tool for PostgreSQL now, Snowflake later.

SQL DIALECTS

Core syntax is ~95% the same across dialects

Dialect	Vendor
PostgreSQL	Open Source (course focus)
T-SQL	Microsoft SQL Server
PL/SQL	Oracle
Snowflake SQL	Snowflake (Week 3)

Debugging & Performance

Reading Errors, Writing Better Queries

READING SQL ERROR MESSAGES

Error messages tell you exactly what's wrong:

```
ERROR: column "cat_name" does not exist
LINE 2: WHERE cat_name = 'Whiskers'
              ^
```

The message tells you: problem, location, and often the fix!

COMMON ERROR PATTERNS

Error Message	Likely Cause
"does not exist"	Typo or wrong name
"must appear in GROUP BY"	Missing column in GROUP BY
"syntax error at or near"	Check punctuation, quotes
"operator does not exist"	Wrong data type comparison

PERFORMANCE BASICS

Filter early (in WHERE):

```
SELECT * FROM cats c
JOIN interactions i ON c.cat_id = i.cat_id
WHERE c.status = 'available';  -- Filter first!
```

Avoid SELECT * in production:

```
SELECT name, status FROM cats;  --
Better!
```

INDEXES: THE TRADE-OFF

Indexes are like a book index — helps find data faster

Help reads: Finding specific rows is fast

Hurt writes: Every INSERT/UPDATE must update the index

Trade-off: More indexes = faster reads, slower writes

SQL GOTCHAS: AVOID THESE MISTAKES

Mistake	Fix
`WHERE name = Whiskers`	`WHERE name = 'Whiskers'`
`WHERE phone = NULL`	`WHERE phone IS NULL`
`WHERE COUNT(*) > 5`	`HAVING COUNT(*) > 5`
Missing semicolon	End statements with ;

SQL STYLE: WRITE READABLE CODE

Common conventions:

- UPPERCASE keywords: SELECT, FROM, WHERE
- lowercase for table/column names
- One clause per line for readability
- Consistent indentation

KEY TAKEAWAYS

1. SQL solves real problems

Eliminates data anomalies

2. The BIG SIX clauses are your foundation

SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY

3. Think declaratively

Tell SQL WHAT you want, not HOW

4. Practice with real data

We'll use the Cat Cafe database all semester

COMING UP: DATABASE DESIGN & JOINS

- WHY separate tables? (Normalization)
- How tables connect (Keys and relationships)
- Crow's Foot Notation for ERD diagrams
- Joining data: INNER, LEFT, RIGHT, FULL
- Window functions